

Разработка real-time интерфейса с поддержкой горизонтального масштабирования WebSocket-соединений для системы отслеживания задач

Защита выпускной квалификационной работы

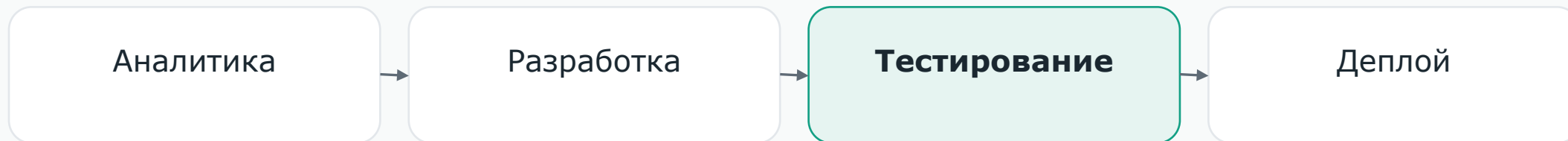
Студент: Зайцев А. С., группа 4131з

Руководитель: канд. техн. наук, доцент А. В. Фомин

Санкт-Петербург, 2026

Где живёт продукт

У команды есть привычный процесс. Продукт помогает на этапе тестирования.



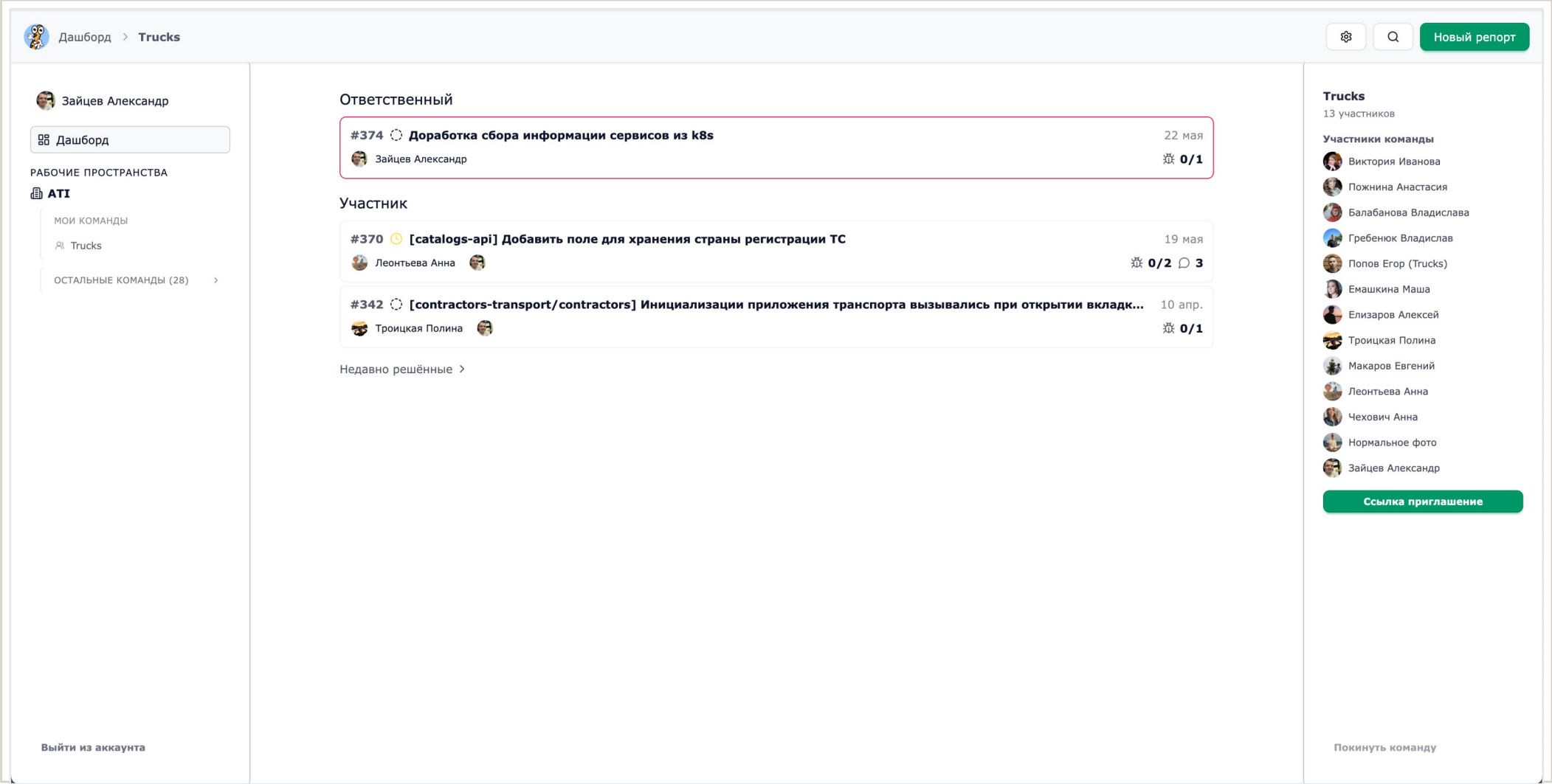
Продукт — система отслеживания задач и багов.

В одном отчёте встречаются тестировщик, разработчик и ответственный: заводят баг, уточняют шаги, меняют статус.

Они работают с одним отчётом одновременно — значит, изменения должны быть видны сразу, без перезагрузки страницы.

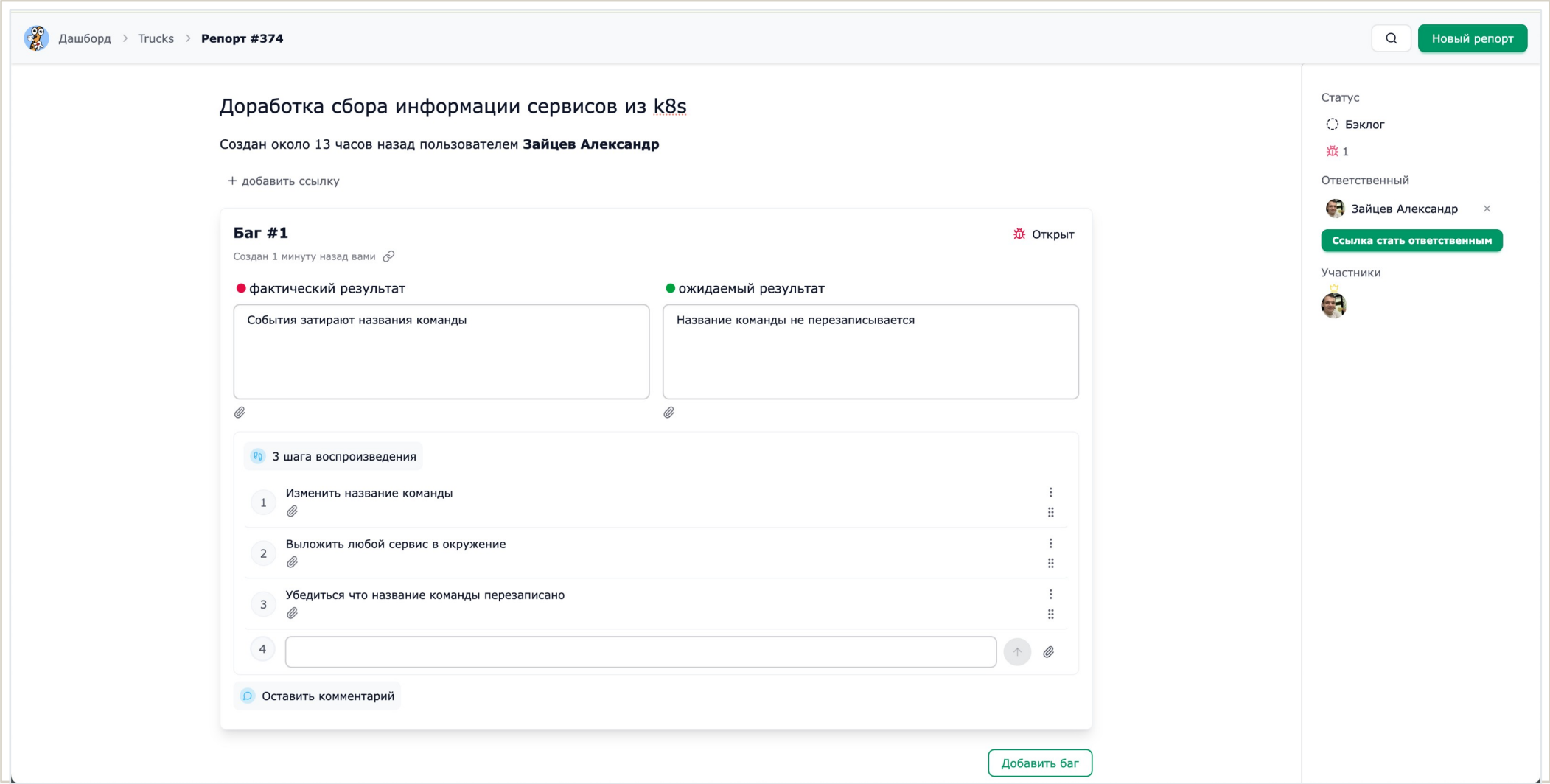
Целевой продукт: дашборд

Список отчётов команды, статусы, ответственные и активность по задачам (рис. 1 ВКР)



Целевой продукт: страница отчёта

Основной рабочий экран: отчёт, баги, шаги воспроизведения, комментарии, участники (рис. 2 ВКР)



Почему это актуально сейчас

Интерфейсы переходят от статичных CRUD-экранов к событийным рабочим пространствам

Продукты усложняются

Пользователь работает не с одной формой, а с отчётом, багами, статусами и участниками одновременно.

AI меняет ожидания от интерфейсов

Взаимодействие всё чаще идёт через диалог с ассистентом: он сам создаёт комментарий, меняет статус, предлагает исполнителя — и пользователь ждёт мгновенного отклика.

Real-time — это инженерная характеристика, а не «фича»

Отзывчивость интерфейса требует масштабируемого и отказоустойчивого контура — на нескольких экземплярах серверного сервиса, а не на одном.

AI-функциональность не утверждается как реализованная — это контекст актуальности.

В чём проблема

Исходный контур реального времени не масштабируется и не отказоустойчив

Не масштабируется

Все клиенты обслуживаются одним серверным процессом. Рост числа пользователей и долгих соединений упирается в один экземпляр; разгрузить второй сервер нельзя.

Не отказоустойчив

Этот единственный процесс — единая точка отказа. При его остановке обновления в реальном времени у всех участников прекращаются, перезапуск приводит к перерыву в работе.

Если просто запустить второй экземпляр, проблема не решается, а смещается: событие, созданное на одном узле, не доходит до клиентов, подключённых ко второму. Совместная работа над отчётом ломается.

Технически: сведения о подключениях и группах SignalR хранятся в памяти конкретного серверного процесса, поэтому без межузлового канала общее пространство синхронизации фрагментируется.

Цель работы

Сделать real-time масштабируемым и отказоустойчивым — с доработкой на всех слоях

Перейти от одноузловой схемы реального времени к распределённому контуру: несколько экземпляров серверного сервиса обслуживают подключения, события об изменении сущностей доставляются всем участникам независимо от узла подключения.

1. Проектирование

Архитектура распределённой доставки, поток события и границы решения.

2. Серверная часть

Диагностика узла, единая точка отправки событий, конфигурируемое подключение общего канала.

3. Клиентская часть

Восстановление соединения, повторная подписка на отчёт после обрыва, диагностические данные.

4. Инфраструктура и испытания

Воспроизводимый стенд из двух экземпляров и общей точки входа; программа испытаний и оценка.

Сравнение подходов к масштабированию

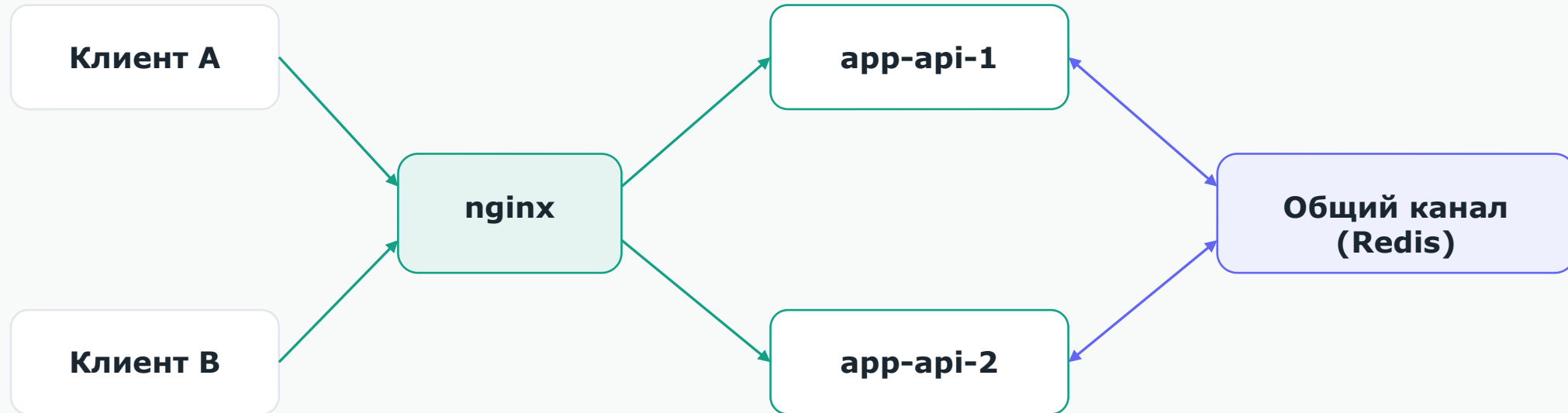
Шесть рассмотренных вариантов — выбран четвёртый (§1.4, таблица 1.4 ВКР)

Подход	Доставка	Ограничение
1. Один экземпляр серверного сервиса	Нет	Только ранний прототип
2. Несколько экземпляров без общего канала	Нет	Локальные группы фрагментируются
3. Закрепление клиента за узлом	Нет	Не доставляет события другим узлам
4. Redis backplane для SignalR — выбрано	Да	Сохраняет SignalR-группы, локальный стенд
5. Брокер сообщений или шина событий	Да	Требует переработки доменных событий
6. Управляемый облачный сервис реального времени	Да	Снижает контроль над локальным контуром

Критерии: межузловая доставка · сохранение модели групп SignalR · self-hosted · без переработки доменных событий · воспроизводимость в Docker Compose

Как это устроено

Два экземпляра backend за общей точкой входа · общий канал событий через Redis



Событие уходит в Redis и приходит на оба экземпляра. Оттуда — клиентам, которые сейчас открыли отчёт.

База данных у экземпляров общая — здесь она опущена, чтобы не отвлекать от канала доставки.

Программная реализация

Пять направлений работ (§4.1 ВКР)

1. Серверная конфигурация

Включаемое через переменную окружения подключение общего канала событий.

2. Хаб и отправка событий

Идентификатор узла, единая точка отправки, диагностика подключения.

3. Клиентская часть

Автоматическое переподключение и повторная подписка на текущий отчёт.

4. Инфраструктура стенда

Два экземпляра серверного сервиса, общая точка входа, общая база и общий канал.

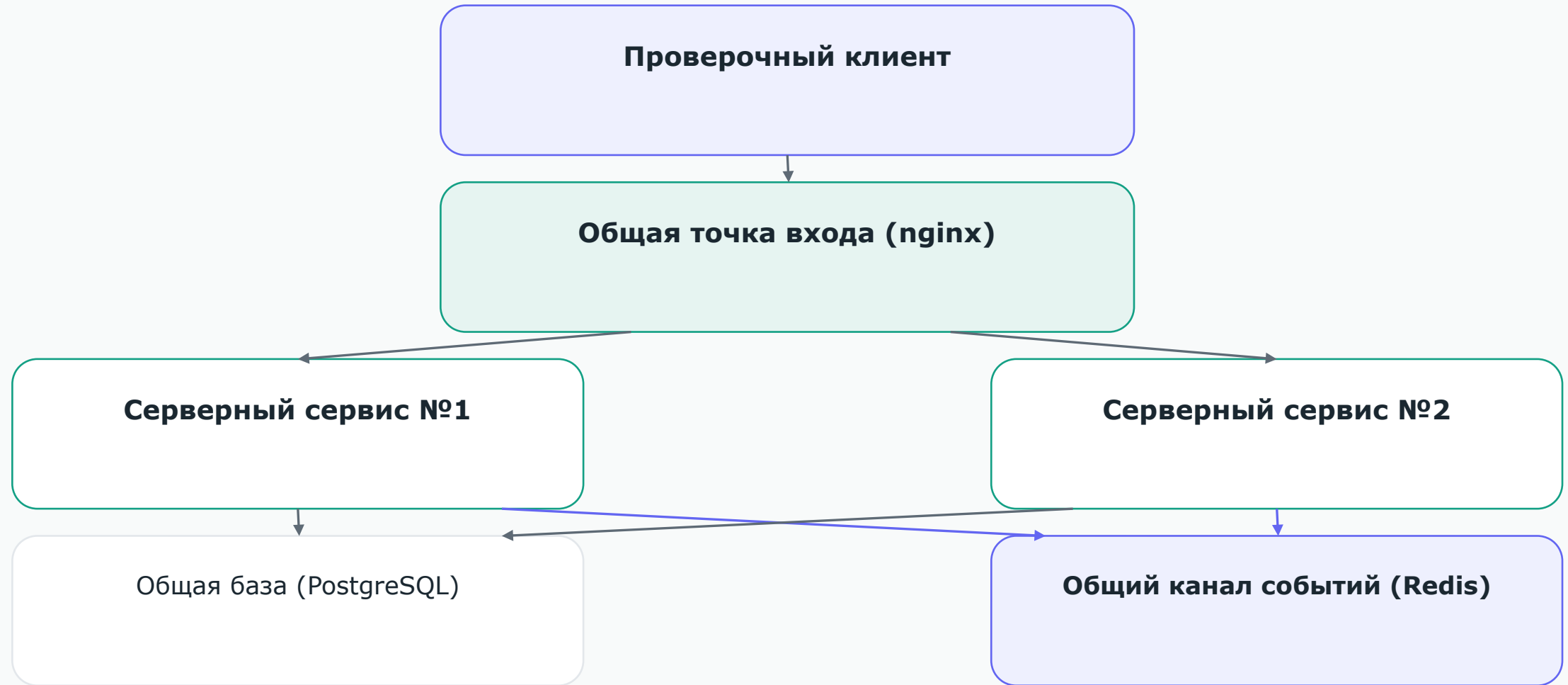
5. Проверочный клиент

Отдельный скрипт, который подключается к двум узлам и измеряет задержку доставки.

Стек: ASP.NET Core, SignalR, Redis, React, TypeScript, Node.js, nginx, Docker Compose

Стенд испытаний

Воспроизводимое локальное окружение из шести компонентов (§4.5 ВКР)



Оба сервиса собраны из одного образа и отличаются только идентификатором экземпляра.

Программа испытаний

Один и тот же стенд, два режима, пять проверок (§5.1 ВКР)

Два режима одного стенда

- Без общего канала событий — контрольный режим, воспроизводит исходное ограничение.
- С общим каналом событий — целевой режим, проверяет решение.

Код и сценарий — одни и те же. Меняется только одна настройка: включён общий канал или нет.

Пять проверок

- 1. Одиночная доставка без общего канала.**
- 2. Одиночная доставка с общим каналом.**
- 3. Серия из 30 повторов — повторяемость и задержка.**
- 4. Восстановление подписки после разрыва соединения.**
- 5. Переключение на другой узел после отказа.**

Сводка результатов

Сопоставление режимов и сценариев (таблица 5.7 ВКР)

Сценарий	Общий канал	Фактический результат	Вывод
Доставка без общего канала	Нет	Тайм-аут 10 000 мс	Ограничение воспроизведено
Доставка с общим каналом	Да	Событие доставлено · задержка ≈ 9,5 мс	Ограничение устранено
Серия из 30 итераций	Да	30 из 30 · 95 % быстрее 13,8 мс	Повторяемость подтверждена
Повторная подписка после разрыва	Да	Событие после восстановления · 15,2 мс	Подписка восстановлена
Переключение после отказа узла	Да	Узел 2 → узел 1, событие получено	Восстановление после отказа

Сопоставление режимов на одном стенде с одним кодом показывает причинную связь между включением межузлового канала и доставкой события.

Чего решение не делает

Это интерфейсная синхронизация, а не строгий промышленный режим реального времени

Задержка не нулевая

Событие проходит через два экземпляра и общий канал.

На стенде получилось ~7 мс в среднем.

Источник истины — база

Если возникает сомнение в состоянии, клиент всегда может перечитать его обычным HTTP-запросом.

Что было бы дальше

Прочный журнал событий, версии сущностей, гарантии доставки — если потребуется строгая согласованность между сервисами.

Итог

Изменения в отчёте теперь доходят до всех участников — независимо от того, к какому экземпляру backend они подключены.

Дополнительный эффект — при падении одного экземпляра пользователи продолжают работать через другой.

Что было

С несколькими экземплярами backend изменения не доходили до части пользователей.

Что сделали

Два экземпляра за общей точкой входа плюс общий канал событий через Redis.

Чем доказали

Сравнение «без / с», 30 повторов, переподключение и падение узла.